# Modelo de Entidades y Relaciones
# Metodología de Sistemas II

**Metodología de sistemas II**

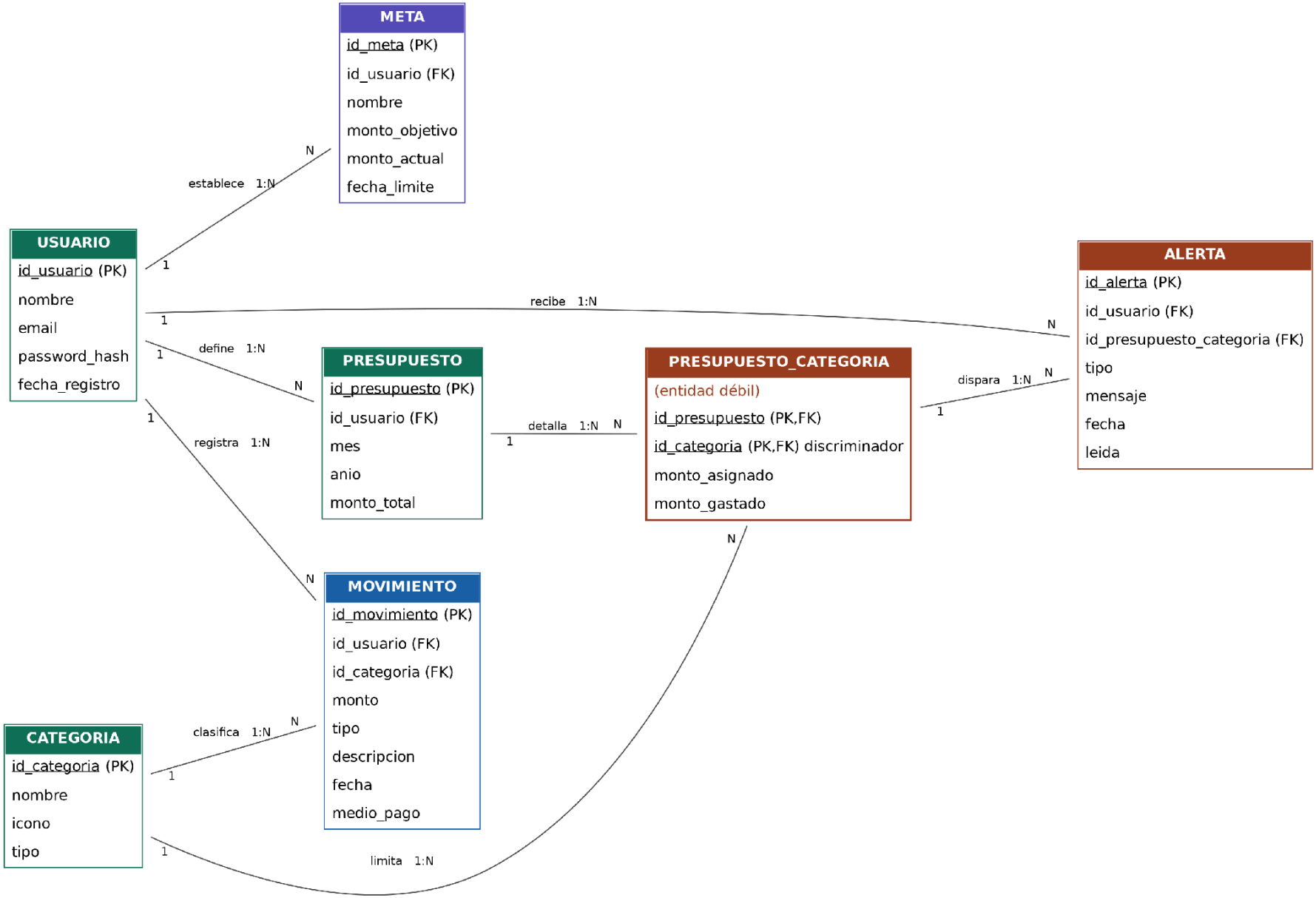
**Cohorte 2025**

**Grupo QWERTY123**

Integrantes: Tomás Andrés, Gustavo Omar Contreras, Paul Romero, Fernando Adaro, Rodriguez Gustavo, Santarena Agustin, Santarena Gabriel, Martin Santiago, Spikerman Nahuel y Jesús Chavez

# Modelo de Entidad-Relación (MER) — GastoSmart

Metodología de Sistemas II — Grupo QWERTY123 — Actividad



# 1. Entidades y atributos

A partir del MVP (resumen, movimientos, agregar gasto, presupuesto y metas) y del Canvas, se identificaron 7 conjuntos de entidades como (USUARIO, PRESUPUESTO, CATEGORIA, etc). Se indica llave primaria (PK), llave foránea (FK) y, en un caso, el discriminador de una entidad débil (PRESUPUESTO_CATEGORIA).

**USUARIO**

| USUARIO | |
|---|---|
| id_usuario | *PK* |
| nombre | |
| email | *único* |
| password_hash | |
| fecha_registro | |

**CATEGORIA**

| CATEGORIA | |
|---|---|
| id_categoria | *PK* |
| nombre | *ej: Alimentación, Transporte, Servicios* |
| icono | |
| tipo | *ingreso / gasto* |

**MOVIMIENTO**

| MOVIMIENTO | |
|---|---|
| id_movimiento | *PK* |
| id_usuario | *FK* |
| id_categoria | *FK* |
| monto | |
| tipo | *ingreso / gasto* |
| descripcion | |
| fecha | |
| medio_pago | *ej: tarjeta de débito* |

**PRESUPUESTO**

| PRESUPUESTO | |
|---|---|
| id_presupuesto | *PK* |
| id_usuario | *FK* |
| mes | |
| anio | |
| monto_total | |

**PRESUPUESTO_CATEGORIA — entidad débil**

*Depende por existencia de PRESUPUESTO: si se elimina un presupuesto, se eliminan sus filas de esta tabla. El discriminador (lo que distingue cada fila dentro de un mismo presupuesto) es id_categoria.*

| PRESUPUESTO_CATEGORIA | |
|---|---|
| id_presupuesto | *PK, FK* |
| id_categoria | *PK, FK — discriminador* |
| monto_asignado | |
| monto_gastado | |

**META**

| META | |
|---|---|
| id_meta | *PK* |
| id_usuario | *FK* |
| nombre | *ej: Viaje a la playa* |
| monto_objetivo | |
| monto_actual | |
| fecha_limite | |

**ALERTA**

| ALERTA | |
|---|---|
| id_alerta | *PK* |
| id_usuario | *FK* |
| id_presupuesto_categoria | *FK* |
| tipo | |
| mensaje | |
| fecha | |
| leida | *booleano* |

## 2. Relaciones y cardinalidades

- USUARIO (1) — registra — (N) MOVIMIENTO
- CATEGORIA (1) — clasifica — (N) MOVIMIENTO
- USUARIO (1) — define — (N) PRESUPUESTO
- PRESUPUESTO (1) — detalla — (N) PRESUPUESTO_CATEGORIA (relación de dependencia por existencia)
- CATEGORIA (1) — limita — (N) PRESUPUESTO_CATEGORIA
- USUARIO (1) — establece — (N) META
- USUARIO (1) — recibe — (N) ALERTA
- PRESUPUESTO_CATEGORIA (1) — dispara — (N) ALERTA

*Todas las relaciones del modelo son uno a muchos (1:N): un usuario tiene muchos movimientos, pero cada movimiento pertenece a un único usuario.*

## 3. Vínculo con n8n

El MER se implementa como tablas en PostgreSQL. n8n se conecta a esa misma base mediante una credencial Postgres y opera sobre las entidades del modelo con dos workflows:

- Alerta de presupuesto: Schedule Trigger → nodo Postgres (SELECT que compara monto_gastado vs monto_asignado en PRESUPUESTO_CATEGORIA) → IF (si se superó el límite) → Send Email/Telegram con el aviso → INSERT en ALERTA.
- Registro de gasto: Webhook (desde la pantalla 'Agregar gasto') → INSERT en MOVIMIENTO → UPDATE de monto_gastado en PRESUPUESTO_CATEGORIA.

De esta forma n8n no es un agregado desconectado del modelo: lee y escribe directamente sobre las tablas que surgen del MER.

## 4. Implementación en PostgreSQL

El script de creación de las 7 tablas se ejecutó sobre la base gasto_smart sin errores:

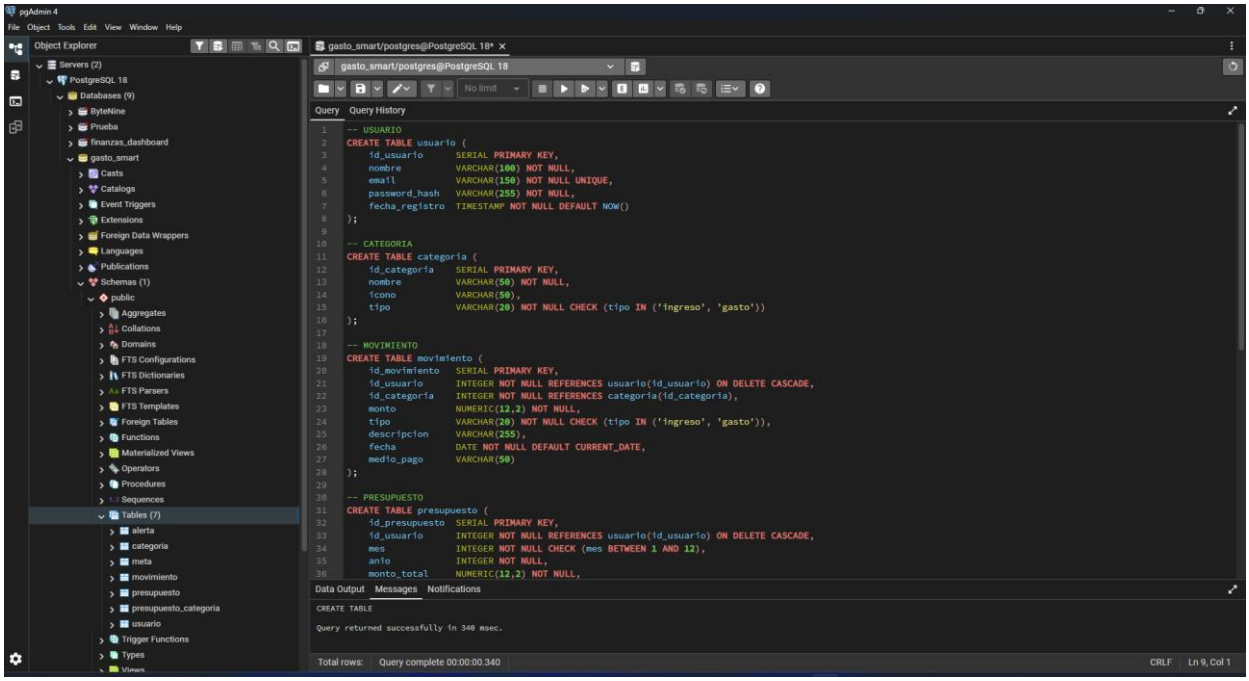



*Figura 1. Árbol de tablas de gasto_smart en pgAdmin4, con el mensaje de ejecución exitosa del script.*

Con datos de prueba cargados, PRESUPUESTO_CATEGORIA muestra el caso que va a disparar la alerta en n8n: la categoría Entretenimiento (id_categoria = 3) tiene monto_gastado (22.000) por encima de monto_asignado (20.000).

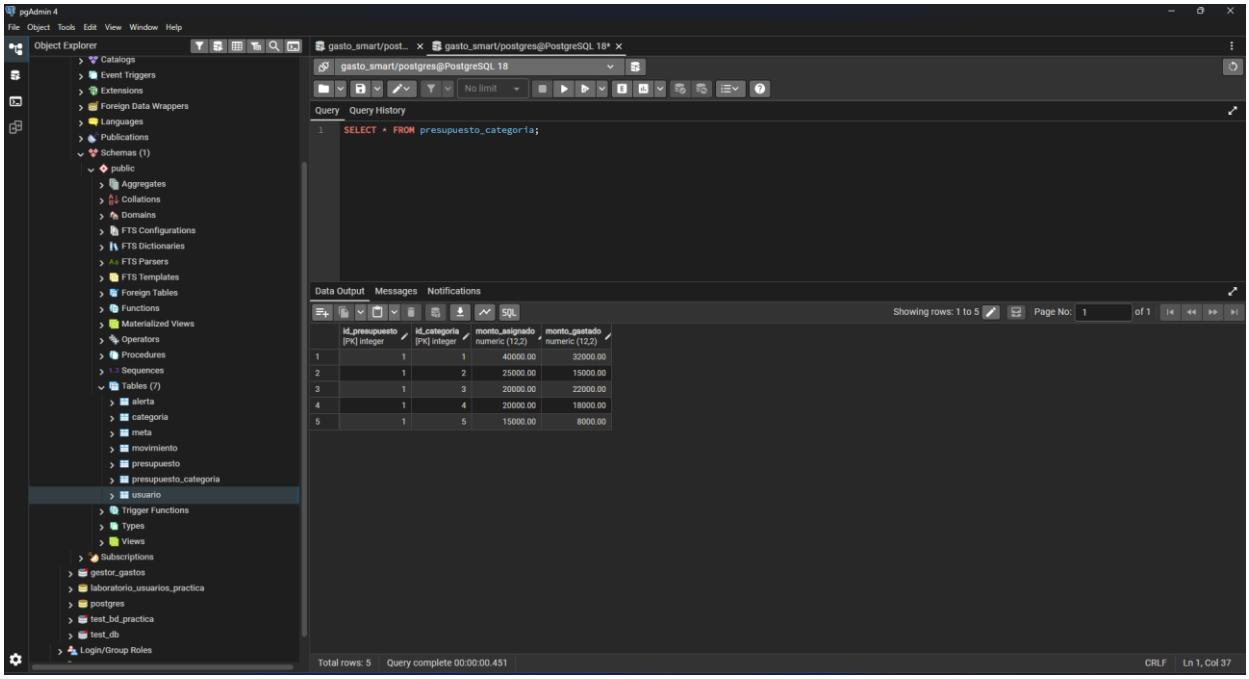



*Figura 2. Datos de PRESUPUESTO_CATEGORIA — en la fila 3 (Entretenimiento), monto_gastado (22.000) supera a monto_asignado (20.000).*

## 5. Implementación en n8n

El workflow de alerta quedó armado con 4 nodos: Schedule Trigger → Execute a SQL query (la consulta contra PRESUPUESTO_CATEGORIA) → If (compara monto_gastado > monto_asignado) → Edit Fields (arma el texto del aviso). Los cuatro ejecutan sin errores y el ítem fluye por la rama true del If, que es la que corresponde al caso cargado.

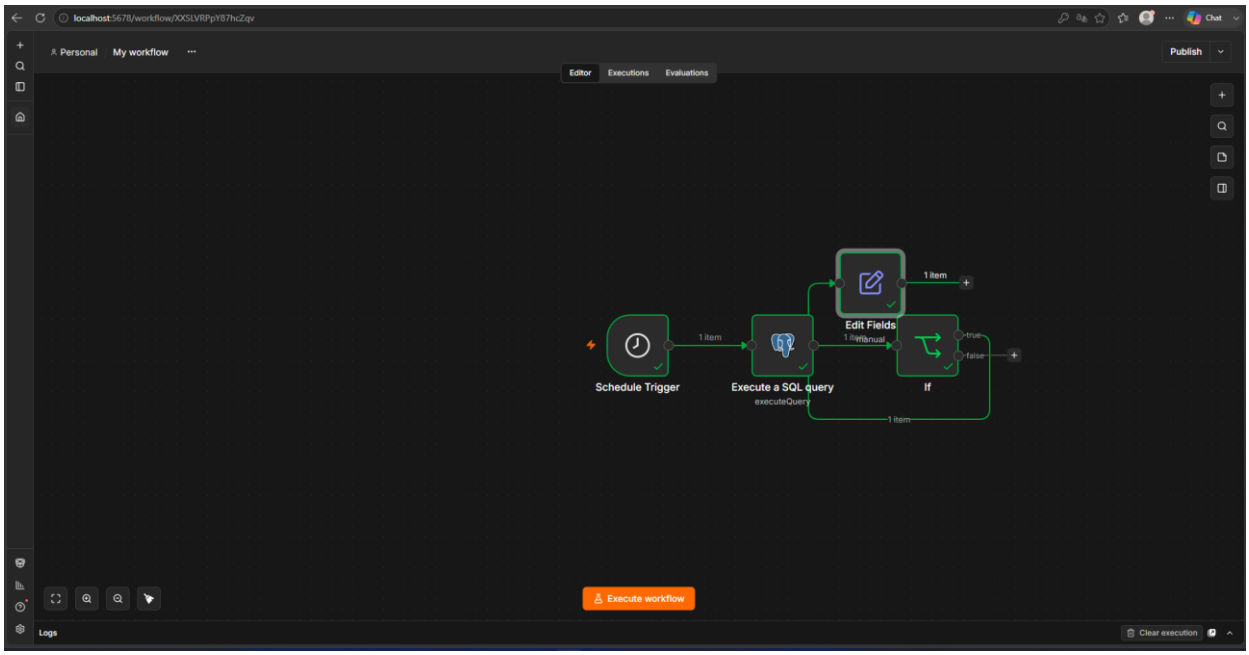



*Figura 3. Workflow completo en n8n: Schedule Trigger → Execute a SQL query → If → Edit Fields, con conexión activa a gasto_smart.*

El nodo Edit Fields resuelve la expresión con los datos reales que trajo la consulta, generando el mensaje de alerta:

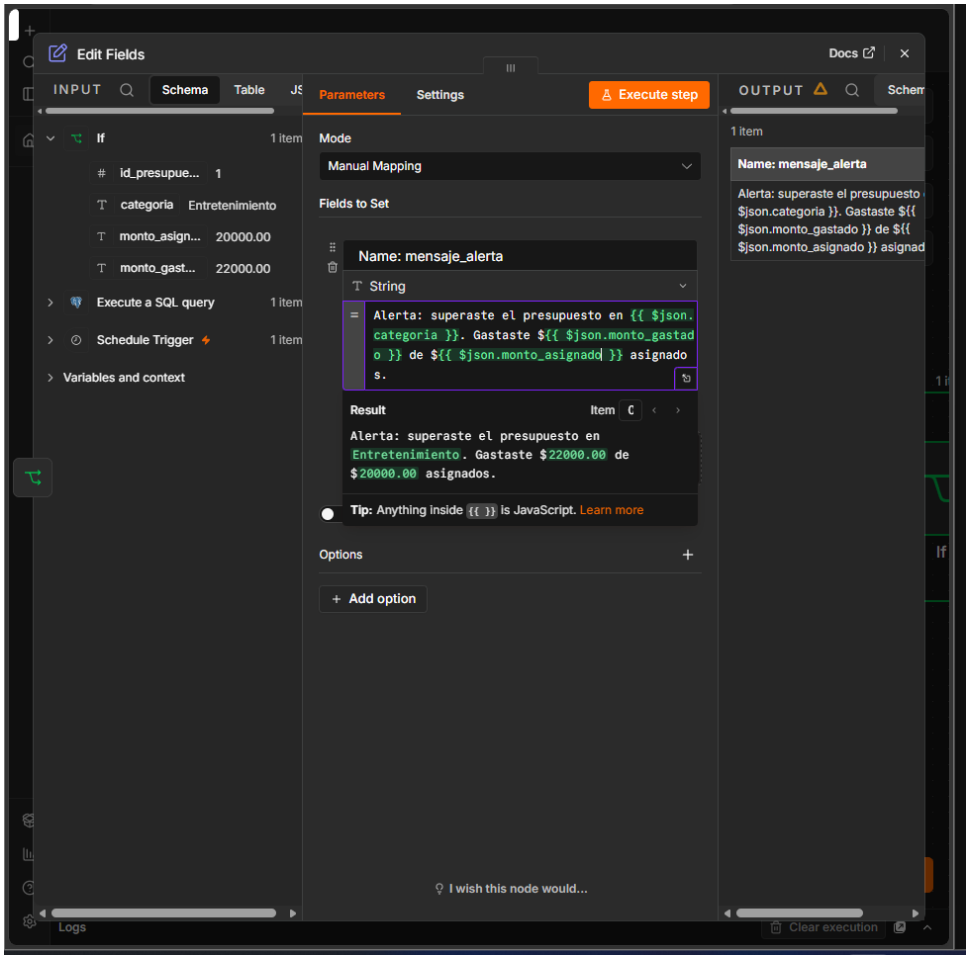



*Figura 4. Mensaje de alerta generado: "Alerta: superaste el presupuesto en Entretenimiento. Gastaste $22000.00 de $20000.00 asignados."*